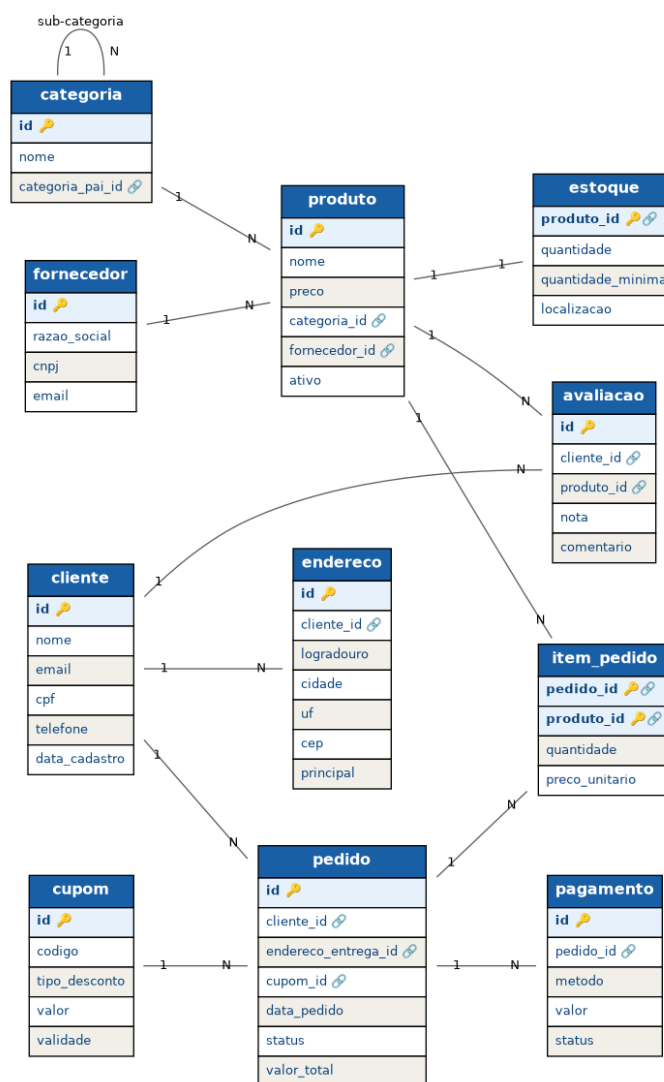


APOSTILA

Banco de Dados Relacional com PostgreSQL

Parte 1 — Fundamentos, Modelagem e Criação do Banco

Estudo de caso: Sistema de E-commerce



Prof. Dr. Anuar José Mincache

Universidade Estadual de Maringá (UEM) · github.com/220719

Sumário

Como usar esta apostila

Capítulo 1 — Fundamentos de Banco de Dados

- 1.1 O que é um banco de dados e um SGBD
- 1.2 O modelo relacional
- 1.3 Por que PostgreSQL
- 1.4 As propriedades ACID

Capítulo 2 — O Projeto: um sistema de e-commerce

- 2.1 Levantamento de requisitos
- 2.2 Regras de negócio

Capítulo 3 — Modelagem Conceitual (MER)

- 3.1 Entidades e atributos
- 3.2 Relacionamentos e cardinalidade
- 3.3 O Diagrama Entidade-Relacionamento

Capítulo 4 — Modelagem Lógica (modelo relacional)

- 4.1 Mapeando o MER em tabelas
- 4.2 A tabela associativa

Capítulo 5 — Normalização aplicada

- 5.1 Anomalias e o problema da tabela única
- 5.2 1FN, 2FN e 3FN passo a passo

Capítulo 6 — Criação do banco (DDL)

- 6.1 CREATE DATABASE e tipos de dados
- 6.2 Criando as tabelas com constraints
- 6.3 Chaves estrangeiras e integridade referencial
- 6.4 Índices

Dicionário de dados

Questões de fixação

Gabarito comentado

Como usar esta apostila

Esta apostila foi escrita para acompanhar, lado a lado, a **teoria** e a **prática** da construção de um banco de dados relacional. Cada conceito teórico é imediatamente aplicado ao nosso estudo de caso — um sistema de e-commerce — e termina virando código SQL que você pode executar no PostgreSQL. A ideia é que, ao final, você não apenas saiba *o que* é uma chave estrangeira, mas tenha *digitado* dezenas delas e entendido o porquê de cada decisão.

Você vai encontrar ao longo do texto algumas caixas destacadas:

◆ No mundo real

Trás situações que aparecem de verdade no dia a dia de quem trabalha com banco de dados — decisões de projeto, armadilhas e como sistemas reais lidam com elas.

★ Dica

Boas práticas e atalhos que tornam seu trabalho mais limpo e profissional.

▲ Atenção

Pontos onde é fácil errar e que merecem atenção redobrada.

× Erro comum

Erros clássicos de quem está começando — mostrados para que você *não* os cometa.

Ao final há **5 questões de múltipla escolha** com gabarito comentado. Cada parte da apostila trará seu próprio conjunto de questões, formando ao final um banco completo para avaliações.

Fundamentos de Banco de Dados

1.1 O que é um banco de dados e um SGBD

Um **banco de dados** é uma coleção organizada de dados relacionados entre si, que representa algum aspecto do mundo real — o que chamamos de *minimundo*. No nosso caso, o minimundo é uma loja virtual: clientes que fazem pedidos de produtos, que por sua vez têm fornecedores, estoque, avaliações e pagamentos.

Esses dados não ficam soltos. Quem os gerencia é o **SGBD** (Sistema Gerenciador de Banco de Dados) — em inglês, *DBMS*. O PostgreSQL é um SGBD. Ele é a camada de software entre você (ou a aplicação) e os arquivos físicos no disco, e é responsável por:

- ▶ **Definir** a estrutura dos dados (quais tabelas existem, quais colunas, quais tipos);
- ▶ **Armazenar e recuperar** dados de forma eficiente, mesmo com milhões de registros;
- ▶ **Controlar o acesso concorrente** — vários usuários lendo e escrevendo ao mesmo tempo sem se atrapalharem;
- ▶ **Garantir a integridade** — impedir, por exemplo, um pedido apontando para um cliente que não existe;
- ▶ **Controlar segurança e permissões** de quem pode ver ou alterar o quê;
- ▶ Fornecer mecanismos de **backup e recuperação** em caso de falha.

◆ No mundo real

Antes dos SGBDs, sistemas guardavam dados em arquivos próprios (planilhas, arquivos de texto, .dbf). O problema clássico era a **redundância**: o endereço do cliente era copiado em cada pedido. Se o cliente mudava de cidade, você precisava caçar e atualizar dezenas de arquivos — e sempre esquecia algum, gerando dados inconsistentes. O modelo relacional nasceu exatamente para atacar esse problema.

1.2 O modelo relacional

Em 1970, Edgar F. Codd, pesquisador da IBM, publicou o artigo que fundou o **modelo relacional**. A ideia central é simples e poderosa: organizar os dados em **relações** — que na prática enxergamos como **tabelas**. Cada tabela tem:

- ▶ **Linhas** (ou *tuplas* / registros): cada uma representa uma ocorrência, por exemplo um cliente específico;
- ▶ **Colunas** (ou *atributos*): as propriedades, como nome, e-mail, CPF;
- ▶ Um **domínio** para cada coluna: o conjunto de valores válidos (a coluna nota de uma avaliação, por exemplo, só aceita inteiros de 1 a 5).

O que torna o modelo *relacional* não é apenas ter tabelas, mas a forma como elas se **relacionam** umas com as outras por meio de **chaves**. É isso que nos permite, sem repetir

dados, responder perguntas como “quais produtos o cliente João comprou no último mês?” — cruzando as tabelas cliente, pedido, item_pedido e produto.

● Sacada

Guarde esta frase: **cada fato deve ser armazenado uma única vez, em um único lugar**. Toda a modelagem e a normalização que veremos são, no fundo, tentativas de honrar esse princípio.

1.3 Por que PostgreSQL

Existem vários SGBDs relacionais (MySQL, SQL Server, Oracle, SQLite). Vamos usar o **PostgreSQL** porque ele é gratuito, de código aberto, extremamente robusto e segue de perto o padrão SQL. Além do básico, ele oferece recursos avançados — tipos JSON, arrays, busca textual, funções, *triggers* — que exploraremos nas próximas partes. É o SGBD preferido em projetos modernos de dados, justamente o que você encontra em vagas de mercado.

1.4 As propriedades ACID

Uma **transação** é um conjunto de operações tratado como uma unidade indivisível. O exemplo clássico é uma compra: dar baixa no estoque, registrar o pagamento e criar o pedido devem acontecer *juntos*. Um SGBD sério garante as quatro propriedades **ACID**:

Propriedade	O que significa	Exemplo no e-commerce
Atomicidade	Ou tudo acontece, ou nada acontece.	Se o pagamento falha, o pedido não é criado nem o estoque baixado.
Consistência	O banco vai de um estado válido a outro válido.	Não sobra um item de pedido apontando para um pedido inexistente.
Isolamento	Transações simultâneas não interferem entre si.	Dois clientes comprando a última unidade: só um leva.
Durabilidade	Uma vez confirmada, a transação persiste mesmo com queda de energia.	Pedido confirmado não “some” se o servidor reiniciar.

O Projeto: um sistema de e-commerce

Toda modelagem começa entendendo o problema. Antes de desenhar qualquer tabela, precisamos conversar com o cliente (ou imaginar o sistema) e levantar **requisitos** e **regras de negócio**. Vamos descrever nosso minimundo.

2.1 Levantamento de requisitos

Nossa loja virtual precisa controlar:

- ▶ **Clientes** cadastrados, cada um com seus dados pessoais e um ou mais **endereços** de entrega;
- ▶ **Produtos** à venda, organizados em **categorias** (que podem ter sub-categorias), fornecidos por **fornecedores**;
- ▶ O **estoque** de cada produto;
- ▶ **Pedidos** feitos pelos clientes, contendo vários produtos, cada um com sua quantidade e preço;
- ▶ **Pagamentos** de cada pedido;
- ▶ **Avaliações** que clientes deixam nos produtos que compraram;
- ▶ **Cupons** de desconto que podem ser aplicados a um pedido.

2.2 Regras de negócio

As regras de negócio são restrições que o sistema deve respeitar. Muitas delas viram *constraints* no banco. Anote-as — vamos implementá-las em SQL no Capítulo 6:

- ▶ RN1: o **e-mail** de um cliente é único no sistema;
- ▶ RN2: um produto pertence a **exatamente uma** categoria e **um** fornecedor;
- ▶ RN3: o **preço** de um produto e a **quantidade** em estoque nunca podem ser negativos;
- ▶ RN4: a **nota** de uma avaliação vai de 1 a 5;
- ▶ RN5: cada item de um pedido guarda o **preço no momento da compra** (o preço do produto pode mudar depois);
- ▶ RN6: o **status** de um pedido só pode ser: pendente, pago, enviado, entregue ou cancelado;
- ▶ RN7: ao apagar um pedido, seus **itens** devem ser apagados junto (eles não existem sem o pedido).

◆ No mundo real

A RN5 parece detalhe, mas é uma das decisões mais importantes do projeto. Se o item de pedido apenas *referenciasse* o preço atual do produto, ao reajustar preços você reescreveria o histórico de todas as vendas passadas — e a contabilidade nunca fecharia. Por isso `item_pedido` tem sua própria coluna `preco_unitario`.

▲ Atenção

A RN7 descreve um relacionamento de **dependência existencial**: um item de pedido é uma **entidade fraca** — não faz sentido sem o pedido ao qual pertence. Isso vai se traduzir num `ON DELETE CASCADE` mais adiante.

Modelagem Conceitual (MER)

O **Modelo Entidade-Relacionamento (MER)**, criado por Peter Chen em 1976, é a forma mais usada para representar visualmente um banco antes de criá-lo. Ele é *independente de SGBD*: descreve **o que** existe, não **como** será implementado. É a planta da casa antes da obra.

3.1 Entidades e atributos

Uma **entidade** é algo do mundo real sobre o qual queremos guardar informações: cliente, produto, pedido. Os **atributos** são as características de cada entidade. Eles se classificam em:

Tipo de atributo	Definição	Exemplo
Simples	Não pode ser subdividido.	nota de uma avaliação
Composto	Pode ser dividido em partes menores.	endereço (logradouro, número, cidade...)
Multivalorado	Pode ter vários valores.	telefone de um cliente
Derivado	Calculado a partir de outros.	valor_total do pedido (soma dos itens)
Identificador	Distingue uma ocorrência das demais (vira chave).	id ou cpf do cliente

★ Dica

Atributos **compostos** e **multivalorados** não existem direto numa tabela relacional. Um atributo composto vira várias colunas; um multivalorado quase sempre vira uma **tabela própria**. Por isso “endereço” virou a entidade endereço — um cliente pode ter vários.

3.2 Relacionamentos e cardinalidade

Um **relacionamento** liga duas (ou mais) entidades. A **cardinalidade** diz *quantas* ocorrências de uma se associam à outra. São três tipos fundamentais:

Cardinalidade	Leitura	No nosso projeto
1:1 (um para um)	Cada A se liga a no máximo um B, e vice-versa.	produto 1:1 estoque
1:N (um para muitos)	Um A se liga a vários B; cada B a um único A.	cliente 1:N pedido
N:M (muitos para muitos)	Vários A se ligam a vários B.	pedido N:M produto

No nosso e-commerce temos exemplos de **todos** os tipos, além de dois casos especiais importantes:

- ▶ **Auto-relacionamento:** categoria se relaciona com ela mesma — uma categoria pode ser sub-categoria de outra (Eletrônicos → Celulares → Smartphones);
- ▶ **Entidade fraca:** `item_pedido` depende da existência de um pedido.

3.3 O Diagrama Entidade-Relacionamento

Juntando entidades, atributos e relacionamentos, chegamos ao diagrama abaixo. Ele já está no formato *físico* (mostrando as chaves), que é a ponte direta para as tabelas do Capítulo 6. O ícone de chave marca a chave primária (PK) e o de corrente marca as chaves estrangeiras (FK).

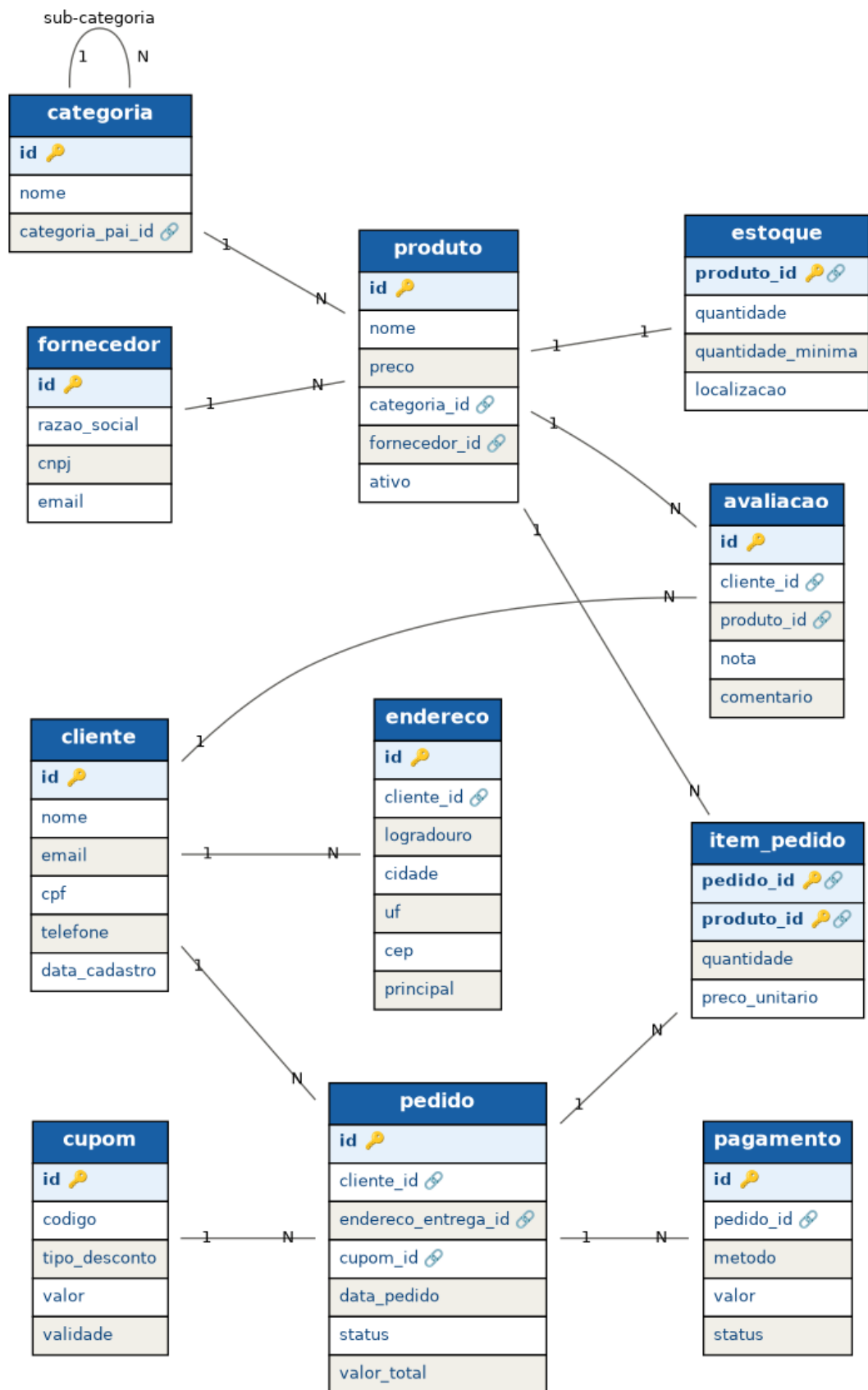


Figura 1 — Diagrama Entidade-Relacionamento do sistema de e-commerce.

Modelagem Lógica (modelo relacional)

O modelo conceitual (MER) é abstrato. Agora vamos transformá-lo no **modelo lógico**: o conjunto de tabelas com suas chaves, ainda independente do PostgreSQL mas já pronto para virar SQL. Esse processo segue regras mecânicas de **mapeamento**.

4.1 Mapeando o MER em tabelas

Situação no MER	Regra de mapeamento
Entidade	Vira uma tabela; atributos viram colunas; o identificador vira a chave primária (PK) .
Relacionamento 1:N	A chave do lado “1” entra como chave estrangeira (FK) na tabela do lado “N”.
Relacionamento 1:1	A FK entra em uma das tabelas (geralmente na de existência opcional), podendo ser também PK.
Relacionamento N:M	Cria-se uma tabela associativa nova, com as FKs das duas entidades.
Auto-relacionamento	A FK referencia a própria tabela .

Aplicando as regras: cliente 1:N pedido faz a coluna `cliente_id` aparecer em pedido. O relacionamento 1:1 produto-estoque faz `produto_id` ser, ao mesmo tempo, PK e FK de estoque. E o auto-relacionamento põe `categoria_pai_id` dentro de categoria.

4.2 A tabela associativa

Um relacionamento N:M **nunca** pode ser representado por uma FK em uma das tabelas — não caberia. Um pedido tem vários produtos, e um produto aparece em vários pedidos. A solução é a tabela associativa `item_pedido`, que fica “entre” as duas:

```
-- item_pedido resolve o N:M entre pedido e produto
-- A PK é composta pelas duas FKs: um produto aparece UMA vez por pedido
item_pedido ( pedido_id -> pedido(id),
  produto_id -> produto(id),
  quantidade,
  preco_unitario ) -- PK = (pedido_id, produto_id)
```

🔍 Sacada

Toda tabela associativa carrega no mínimo as duas FKs. Mas ela quase sempre ganha **atributos próprios** — aqui, `quantidade` e `preco_unitario`. Esses atributos “pertencem ao relacionamento”: a quantidade não é do pedido nem do produto isoladamente, e sim da combinação dos dois.

Normalização aplicada

Normalizar é organizar as colunas e tabelas para eliminar redundância e evitar **anomalias**. Vamos partir de um projeto *ruim* de propósito — uma única tabela guardando tudo — e consertá-lo passo a passo até a 3FN.

5.1 Anomalias e o problema da tabela única

Imagine guardar todos os pedidos numa tabela só, repetindo os dados do cliente e do produto em cada linha:

pedido	cliente	cidade	produto1	produto2	valor
101	Ana Souza	Maringá	Mouse; 2; 50	Teclado; 1; 120	220
102	Ana Souza	Maringá	Monitor; 1; 900	—	900

Esse desenho gera três tipos de problema:

- ▶ **Anomalia de inserção:** para cadastrar um produto novo, sou obrigado a ter um pedido. Não consigo registrar um produto “avulso”;
- ▶ **Anomalia de atualização:** se Ana muda de cidade, preciso alterar *todas* as linhas dela — esquecendo uma, o dado fica inconsistente;
- ▶ **Anomalia de exclusão:** se eu apagar o último pedido de um produto, perco a informação de que o produto existe.

× Erro comum

Colunas como produto1, produto2, produto3... são um sintoma clássico de projeto não normalizado. Toda vez que você pensar em numerar colunas, pare: aquilo é uma **nova tabela** esperando para nascer.

5.2 1FN, 2FN e 3FN passo a passo

Primeira Forma Normal (1FN) — cada célula deve conter um único valor (atômico); não pode haver grupos repetidos nem listas dentro de uma célula. Quebramos a célula “Mouse; 2; 50” em linhas separadas, criando a base de uma tabela de itens:

```
-- Apos a 1FN: uma linha por item, valores atomicos
(101, Ana Souza, Maringa, Mouse, 2, 50)
(101, Ana Souza, Maringa, Teclado, 1, 120)
(102, Ana Souza, Maringa, Monitor, 1, 900)
```

Segunda Forma Normal (2FN) — estando em 1FN, nenhum atributo não-chave pode depender apenas de *parte* da chave (dependência parcial). A chave aqui é (pedido, produto). Mas o nome e a cidade do cliente dependem só do pedido, não do produto. Separamos:

```
-- 2FN: dados do pedido saem para sua propria tabela
pedido (101, Ana Souza, Maringa)
item (101, Mouse, 2, 50)
item (101, Teclado, 1, 120)
```

Terceira Forma Normal (3FN) — estando em 2FN, nenhum atributo não-chave pode depender de outro atributo não-chave (dependência transitiva). A cidade depende do *cliente*, não do *pedido*. Logo, o cliente vira sua própria tabela:

```
-- 3FN: cliente isolado; pedido guarda so a FK do cliente
cliente (1, Ana Souza, Maringa)
pedido (101, cliente_id=1)
item (101, produto_id, quantidade, preco)
```

◆ No mundo real

Repare que a normalização nos levou **naturalmente** ao mesmo desenho que projetamos no MER: tabelas separadas de cliente, pedido e item. Modelagem e normalização são dois caminhos para o mesmo objetivo. Na prática, projeta-se bem no MER e usa-se a normalização para *conferir*.

★ Dica

Para a 3FN existe uma frase mnemônica famosa: cada atributo não-chave deve depender “*da chave, da chave inteira e de nada além da chave*”. As três partes da frase são, na ordem, 1FN, 2FN e 3FN.

Criação do banco (DDL)

Chegou a hora de transformar tudo em código. A **DDL** (*Data Definition Language*) é a parte do SQL que **define** estruturas: CREATE, ALTER, DROP. Vamos do banco vazio até todas as tabelas, constraints e índices.

6.1 CREATE DATABASE e tipos de dados

```
-- Cria o banco com codificacao UTF-8 (acentos!) e collation pt-BR
CREATE DATABASE ecommerce
WITH ENCODING = 'UTF8'
LC_COLLATE = 'pt_BR.UTF-8'
LC_CTYPE = 'pt_BR.UTF-8'
TEMPLATE = template0;

-- Em seguida, conecte-se a ele no psql:
\c ecommerce
```

▲ Atenção

O `\c` é um comando do cliente `psql`, não do SQL. Sem se conectar ao banco novo, você criaria as tabelas no banco errado (o postgres padrão). É um tropeço clássico de quem está começando.

Os principais tipos de dados que vamos usar:

Tipo PostgreSQL	Uso	Exemplo
SERIAL / IDENTITY	Inteiro auto-incrementado (chaves).	id
VARCHAR(n)	Texto com limite de tamanho.	nome VARCHAR(120)
TEXT	Texto longo, sem limite.	comentario
NUMERIC(p,s)	Decimal exato (dinheiro!).	preco NUMERIC(10,2)
INTEGER	Número inteiro.	quantidade
BOOLEAN	Verdadeiro/falso.	ativo
DATE / TIMESTAMP	Data / data e hora.	data_pedido

× Erro comum

Nunca use `FLOAT` ou `REAL` para dinheiro. São tipos de ponto flutuante e sofrem erros de arredondamento (0,1 + 0,2 não dá exatamente 0,3). Para valores monetários, sempre `NUMERIC(p,s)`, que é exato.

6.2 Criando as tabelas com constraints

Vamos criar as tabelas na ordem correta: primeiro as que não dependem de ninguém, depois as que as referenciam. Começamos por cliente, aplicando as regras de negócio

RN1 (e-mail único) que levantamos no Capítulo 2:

```
CREATE TABLE cliente (  
  id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  nome VARCHAR(120) NOT NULL,  
  email VARCHAR(150) NOT NULL UNIQUE, -- RN1  
  cpf CHAR(11) NOT NULL UNIQUE,  
  telefone VARCHAR(20),  
  data_cadastro TIMESTAMP NOT NULL DEFAULT now()  
);
```

Vejamos cada **constraint** (restrição) usada:

- ▶ PRIMARY KEY: identifica unicamente a linha; implica NOT NULL + UNIQUE;
- ▶ NOT NULL: a coluna é obrigatória;
- ▶ UNIQUE: não admite valores repetidos (dois clientes não têm o mesmo e-mail);
- ▶ DEFAULT now(): se o valor não for informado, usa a data/hora atual;
- ▶ GENERATED ALWAYS AS IDENTITY: forma moderna (padrão SQL) de auto-incremento, preferível ao antigo SERIAL.

Agora a entidade categoria, que traz nosso **auto-relacionamento**: a coluna categoria_pai_id referencia a própria tabela (e é NULL nas categorias de topo):

```
CREATE TABLE categoria (  
  id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  nome VARCHAR(80) NOT NULL,  
  categoria_pai_id INTEGER,  
  CONSTRAINT fk_categoria_pai  
  FOREIGN KEY (categoria_pai_id) REFERENCES categoria(id)  
);
```

A tabela produto implementa a RN2 (uma categoria e um fornecedor) e a RN3 (preço não-negativo, via CHECK):

```
CREATE TABLE produto (  
  id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  nome VARCHAR(150) NOT NULL,  
  descricao TEXT,  
  preco NUMERIC(10,2) NOT NULL CHECK (preco >= 0), -- RN3  
  ativo BOOLEAN NOT NULL DEFAULT TRUE,  
  categoria_id INTEGER NOT NULL REFERENCES categoria(id), -- RN2  
  fornecedor_id INTEGER NOT NULL REFERENCES fornecedor(id) -- RN2  
);
```

O relacionamento **1:1** com estoque aparece de forma elegante: a chave primária de estoque é *também* a chave estrangeira para produto. Sendo PK, ela é única — e isso é exatamente o que garante o “um para um”:

```
CREATE TABLE estoque (
  produto_id INTEGER PRIMARY KEY REFERENCES produto(id),
  quantidade INTEGER NOT NULL DEFAULT 0 CHECK (quantidade >= 0),
  quantidade_minima INTEGER NOT NULL DEFAULT 0,
  localizacao VARCHAR(40)
);
```

● Sacada

Por que o 1:1 vira PK+FK na *mesma* coluna? Porque a PK garante que cada produto aparece no máximo **uma vez** em estoque. Se fosse apenas FK, nada impediria dois registros de estoque para o mesmo produto — e aí já não seria 1:1, e sim 1:N.

6.3 Chaves estrangeiras e integridade referencial

A tabela pedido referencia cliente, endereço e cupom, e implementa a RN6 (status válidos) com um CHECK:

```
CREATE TABLE pedido (
  id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  cliente_id INTEGER NOT NULL REFERENCES cliente(id),
  endereco_entrega_id INTEGER NOT NULL REFERENCES endereco(id),
  cupom_id INTEGER REFERENCES cupom(id), -- opcional (NULL)
  data_pedido TIMESTAMP NOT NULL DEFAULT now(),
  status VARCHAR(15) NOT NULL DEFAULT 'pendente'
  CHECK (status IN ('pendente', 'pago', 'enviado',
    'entregue', 'cancelado')), -- RN6
  valor_total NUMERIC(10,2) NOT NULL DEFAULT 0
);
```

Agora a associativa item_pedido. Ela tem **chave primária composta** e implementa a RN7 com ON DELETE CASCADE: ao apagar o pedido, seus itens vão junto. Já a FK para produto usa ON DELETE RESTRICT — não deixa apagar um produto que está em pedidos:

```
CREATE TABLE item_pedido (
  pedido_id INTEGER NOT NULL
  REFERENCES pedido(id) ON DELETE CASCADE, -- RN7
  produto_id INTEGER NOT NULL
  REFERENCES produto(id) ON DELETE RESTRICT,
  quantidade INTEGER NOT NULL CHECK (quantidade > 0),
  preco_unitario NUMERIC(10,2) NOT NULL CHECK (preco_unitario >= 0),
  PRIMARY KEY (pedido_id, produto_id) -- chave composta
);
```

Ação ON DELETE	O que faz ao apagar a linha referenciada	Quando usar
CASCADE	Apaga em cascata as linhas filhas.	Entidade fraca (item de pedido).
RESTRICT	Impede a exclusão se houver filhas.	Proteger dados em uso (produto vendido).
SET NULL	Coloca NULL na FK das filhas.	Relação opcional (cupom de um pedido).

◆ No mundo real

Escolher a ação de ON DELETE errada é fonte de bugs sérios em produção. Um CASCADE descuidado numa tabela central pode apagar meio banco com um único DELETE. A regra de ouro: **CASCADE só para entidades que realmente não existem sem o pai.**

6.4 Índices

Toda PK e toda coluna UNIQUE já ganham um índice automaticamente. Mas as **chaves estrangeiras não** — e é por elas que mais cruzamos tabelas (JOIN). Criar índices nas FKs mais usadas acelera bastante as consultas:

```
CREATE INDEX idx_pedido_cliente ON pedido(cliente_id);
CREATE INDEX idx_item_produto ON item_pedido(produto_id);
CREATE INDEX idx_produto_categoria ON produto(categoria_id);
CREATE INDEX idx_avaliacao_produto ON avaliacao(produto_id);
```

★ Dica

Índice não é de graça: ele acelera leituras mas **deixa as escritas um pouco mais lentas** e ocupa disco, pois precisa ser atualizado a cada INSERT/UPDATE. Indexe as colunas que aparecem em WHERE e JOIN — não todas, indiscriminadamente.

Com isso, o banco está estruturalmente completo. O script SQL integral acompanha esta apostila no repositório (pasta sql/), pronto para rodar. Na Parte 2 vamos populá-lo com dados (DML) e começar a consultá-lo (DQL).

Dicionário de dados (resumo)

O dicionário de dados documenta cada tabela e suas chaves — é a referência que toda equipe consulta. Abaixo, a visão geral das 12 tabelas do projeto:

Tabela	PK	FKs	Papel
cliente	id	—	Quem compra.
endereco	id	cliente_id	Endereços de entrega (1:N).
categoria	id	categoria_pai_id	Hierarquia de categorias (auto-rel.).
fornecedor	id	—	Quem fornece os produtos.
produto	id	categoria_id, fornecedor_id	Itens à venda.
estoque	produto_id	produto_id	Quantidade por produto (1:1).
pedido	id	cliente_id, endereco_entrega_id, cupom_id	Compras realizadas.
item_pedido	(pedido_id, produto_id)	pedido_id, produto_id	Associativa pedido–produto (N:M).
pagamento	id	pedido_id	Pagamentos de um pedido.
avaliacao	id	cliente_id, produto_id	Notas e comentários (N:M).
cupom	id	—	Descontos aplicáveis.

Questões de fixação

Marque a única alternativa correta. O gabarito comentado vem logo a seguir.

1. No nosso modelo, o relacionamento entre produto e estoque foi implementado fazendo `produto_id` ser, ao mesmo tempo, chave primária e estrangeira de estoque. Que cardinalidade isso garante?

- a) 1:N (um produto, vários estoques)
- b) N:M (vários produtos, vários estoques)
- c) 1:1 (um produto, no máximo um registro de estoque)
- d) Nenhuma, pois FK não pode ser PK

2. Por que a coluna `preco` de dinheiro deve usar `NUMERIC(10,2)` e não `FLOAT`?

- a) Porque `NUMERIC` ocupa menos espaço em disco
- b) Porque `FLOAT` é ponto flutuante e sofre erros de arredondamento
- c) Porque `FLOAT` não aceita valores negativos
- d) Não há diferença prática entre os dois

3. Uma tabela está na 1FN e na 2FN. A coluna `cidade` depende da coluna `cliente`, que não é chave. Que forma normal está sendo violada e qual a solução?

- a) Viola a 1FN; tornar a célula atômica
- b) Viola a 2FN; remover dependência parcial
- c) Viola a 3FN; remover a dependência transitiva criando a tabela cliente
- d) Não viola nenhuma forma normal

4. Na tabela `item_pedido`, a FK para `pedido` usa `ON DELETE CASCADE`. Qual é o efeito disso?

- a) Ao apagar um item, o pedido inteiro é apagado
- b) Ao apagar um pedido, todos os seus itens são apagados automaticamente
- c) Impede que o pedido seja apagado enquanto tiver itens
- d) Coloca `NULL` no `pedido_id` dos itens

5. Qual estrutura é a forma correta de representar o relacionamento muitos-para-muitos entre `pedido` e `produto`?

- a) Uma coluna `produto_id` na tabela `pedido`
- b) Colunas `produto1`, `produto2`, `produto3` na tabela `pedido`
- c) Uma tabela associativa (`item_pedido`) com as FKs das duas entidades
- d) Uma coluna `pedido_id` na tabela `produto`

Gabarito comentado

1C

Sendo `produto_id` a chave primária de estoque, ela é obrigatoriamente única. Logo, cada produto aparece no máximo uma vez — a definição de **1:1**. Se fosse apenas FK (sem ser PK), poderiam existir vários estoques para o mesmo produto, virando 1:N. A alternativa (d) está errada: uma FK *pode* sim ser PK.

2B

FLOAT/REAL são tipos de ponto flutuante binário e não representam exatamente frações decimais — cálculos com dinheiro acumulam erros de centavos. NUMERIC(p, s) é decimal exato. Sempre use-o para valores monetários.

3C

Dependência de um atributo não-chave em *outro* atributo não-chave é uma **dependência transitiva**, violando a **3FN**. A solução é extrair `cliente` (e sua cidade) para uma tabela própria, deixando no pedido apenas a FK `cliente_id`. A (b) descreve a 2FN (dependência *parcial* da chave), que é um caso diferente.

4B

ON DELETE CASCADE na FK que aponta para pedido faz com que, ao apagar um pedido, todas as linhas filhas em `item_pedido` sejam removidas em cascata. É o comportamento correto para uma **entidade fraca** (item não existe sem o pedido). A opção (c) descreveria o RESTRICT; a (d), o SET NULL.

5C

Um relacionamento **N:M** exige uma **tabela associativa** contendo as chaves estrangeiras das duas entidades (e, opcionalmente, atributos próprios como quantidade e preço). As opções (a) e (d) só conseguiriam modelar 1:N; a (b) é o clássico antipadrão de colunas repetidas, que viola a 1FN.

Fim da Parte 1. Na Parte 2: inserção de dados (DML), transações e o início das consultas com SELECT. O repositório com todos os scripts SQL está em github.com/220719.